CHAPTER 34 · BONUS IMPLEMENTATION GUIDE

Implementation & Deployment Guide

Step-by-Step Lab Setup for the Complete DevSecOps Platform

13	72–104	30 min
Deployment Phases	Total Hours	Rapid-Deploy Option

CHAPTER GOAL

Build a Production-Ready Lab, Phase by Phase

What this chapter delivers

- A detailed, step-by-step guide to deploy the complete DevSecOps platform from scratch.
- Organized into 13 phases — foundation, automation, security, CI/CD, containers, and advanced capabilities.
- Includes validation checks, customization steps, troubleshooting, migration, and disaster recovery.
- Follow it top to bottom for a full build, or jump to the Rapid Deployment option for a 30-minute stand-up.

Time Investment

72–104

total hours across all phases

Or deploy the entire platform in **30 minutes** using the rapid-deploy script (§34.4).

Hardware Requirements

CPU

56+ cores

can start with 32

RAM

94 GB

can start with 64 GB

DISK

890 GB SSD

can start with 500 GB

NETWORK

10 Gbps

1 Gbps acceptable

Hypervisor Stack

- KVM / QEMU on Ubuntu 22.04 LTS
- libvirt for VM management
- Nested virtualization enabled (required if running in the cloud)

Knowledge & Software Prerequisites

Knowledge Requirements

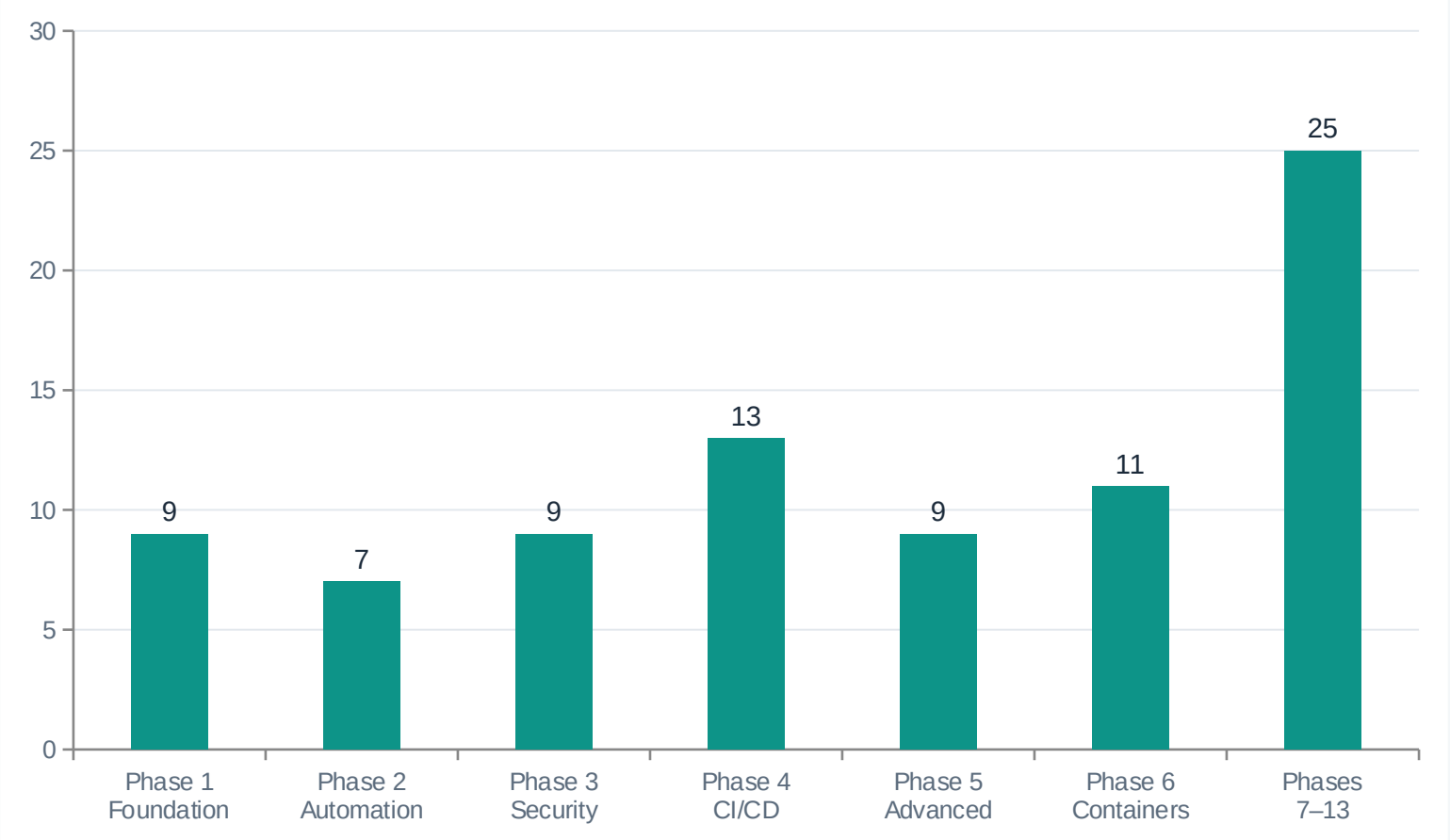
- Linux administration (intermediate)
- Networking fundamentals
- Container concepts
- Kubernetes basics
- Infrastructure as Code

Software Baseline

- Ubuntu 22.04 LTS base image
- Docker & containerd
- Kubernetes 1.28+
- Terraform / OpenTofu

Tip: provision the Packer golden image early — Phase 2 depends on it.

Phase-by-Phase Time Estimates



Total Build Time

72–104

hours, solo-operator pace

Longest phase: **Phase 4, CI/CD** at 12–14 hours.

Prefer speed? See the 30-minute rapid-deploy path.

34.2 PHASE 1

Foundation Deployment — 8–10 Hours

1.1



Host Preparation

Update the OS, install the hypervisor stack, enable nested virtualization, define the lab network, and build the Ubuntu base image.

1.2



Deploy Foundation VMs

Use Terraform to stand up DNS, Identity, and Edge VMs, then apply the Phase 1 Ansible playbook to configure them.

1.3



Validation

Confirm DNS resolution, identity service health, edge routing, and HTTPS/TLS are all working before moving on.

Host Preparation

Update system packages, install the KVM/QEMU hypervisor stack, enable nested virtualization, and create the lab network before any VM is built.

- Update & upgrade the base OS
- Install qemu-kvm, libvirt, virt-manager, terraform, ansible
- Enable nested virtualization via kvm_intel module options
- Define & start the virbr-lab NAT network (10.0.2.0/24)
- Build the ubuntu-base golden VM with virt-install

```
#!/bin/bash
# 01-host-preparation.sh

sudo apt-get update && sudo apt-get upgrade -y

sudo apt-get install -y \
  qemu-kvm libvirt-daemon-system \
  libvirt-clients virt-manager \
  terraform ansible

# Enable nested virtualization
cat <<EOF | sudo tee \
  /etc/modprobe.d/kvm-nested.conf
options kvm_intel nested=1
options kvm_intel ept=1
EOF

sudo virsh net-start virbr-lab
sudo virsh net-autostart virbr-lab
```

Deploy Foundation VMs

DNS, Identity, and Edge — brought up with Terraform, then configured with Ansible.

```
#!/bin/bash
# 01-deploy-foundation.sh

cd ~/lab/opentofu/devsecops-lab
terraform init

# Deploy only Phase 1 VMs (DNS / Identity / Edge)
terraform apply \
  -target='libvirt_domain.foundation' \
  -var="deploy_phase=1"

sleep 60   # allow VMs to boot

cd ~/lab/ansible/devsecops-lab
ansible-playbook -i inventory.ini playbooks/phase-1.yml

ansible all -m ping   # verify reachability
```

Validation Checklist

Run these four checks before declaring Phase 1 complete.

DNS Resolution

```
dig @10.0.2.10 devsecops.com +short
```

Identity Service

```
curl -k https://auth.devsecops.com/health
```

Edge Routing

```
curl -k https://10.0.2.12/health
```

TLS / HTTPS

```
openssl s_client -connect 10.0.2.12:443
```

Automation Deployment — 6–8 Hours

- Build the Packer golden image
- Deploy Phase 2 VMs with Terraform
- Configure with the Phase 2 Ansible playbook
- Run the Phase 2 validation playbook



Depends on Phase 1. Foundation VMs (DNS/Identity/Edge) must be healthy first.

```
#!/bin/bash
# 02-deploy-automation.sh

# Build Packer golden image
cd ~/lab/packer
packer build \
  -var="qemu_binary=/usr/bin/qemu-system-x86_64" \
  ubuntu-22.04/ubuntu-22.04.pkr.hcl

# Deploy Phase 2 VMs
cd ~/lab/opentofu/devsecops-lab
terraform apply \
  -target='libvirt_domain.automation' \
  -var="deploy_phase=2"

# Configure & verify
cd ~/lab/ansible/devsecops-lab
ansible-playbook playbooks/phase-2.yml
ansible-playbook playbooks/validate/phase-2.yml
```

34.4 RAPID DEPLOYMENT OPTION

Deploy Everything in 30 Minutes

1

Clone repositories

git clone the full lab repo

→

2

Generate variables

Write terraform.tfvars with deploy_all_phases = true

→

3

Terraform apply

init → plan → apply the full stack

→

4

Ansible site.yml

Configure every host from inventory.ini

→

5

Validate & inspect

Run validate/all.yml, then kubectl get nodes

```
$ terraform apply tfplan  &&  ansible-playbook playbooks/site.yml -i inventory.ini  &&  kubectl get nodes
```

34.5 CUSTOMIZATION GUIDE

Adapt the Lab to Your Environment

Network CIDR

sed -i the 10.0.2.0/24 default in opentofu/variables.tf

Domain name

Replace devsecops.com in the DNS zone templates

Worker node count

Edit worker_count in opentofu/vms.tf to scale out

Compute resources

Adjust CPU / RAM / disk sizing in opentofu/vms.tf

Container registry

Point manifests at your-registry.com instead of Harbor

Common Issues & Diagnostic Commands

VMs not starting

```
$ virsh list --all
$ virsh start vm-name
```

Network issues

```
$ virsh net-list
$ virsh net-dumpxml virbr-lab
```

DNS not resolving

```
$ nslookup devsecops.com 10.0.2.10
```

Kubernetes not ready

```
$ kubectl get nodes -o wide
$ kubectl describe node cp-01
```

Terraform state issues

```
$ terraform state list
$ terraform state rm resource.name
```

Ansible inventory issues

```
$ ansible all -i inventory.ini -m
ping
```

From Existing Infrastructure

1. Inventory existing systems with ansible-inventory
2. Map existing systems to the DevSecOps architecture
3. Import existing VMs with terraform import
4. Repoint DNS to the new infrastructure
5. Run both systems in parallel for 2 weeks & monitor
6. Decommission old systems with terraform destroy

Backup & Recovery Procedures

- Create backup directories for configs, VMs, and databases
- Export VM definitions with virsh dumpxml
- Snapshot etcd via etcdctl snapshot save
- Snapshot Vault's Raft storage
- Periodically test recovery with virsh create



Test recovery quarterly, not just after an incident.

```
#!/bin/bash
# Setup backup and recovery

mkdir -p /mnt/backups/{configs,vms,databases}

virsh dumpxml cp-01 > \
  /mnt/backups/vms/cp-01.xml

ETCDCTL_API=3 etcdctl snapshot save \
  /mnt/backups/databases/etcd.db

vault write sys/storage/raft/snapshot \
  file=@/mnt/backups/databases/vault-snapshot

# Test recovery
virsh create /mnt/backups/vms/cp-01.xml
```

READY TO BUILD

From Zero to Production-Ready Lab

- Plan hardware & software prerequisites (§34.1)
- Deploy Foundation, then Automation (§34.2–34.3)
- Or take the 30-minute rapid-deploy path (§34.4)
- Customize, troubleshoot, migrate, and back up (§34.5–34.8)